

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4522 — Ağ Tabanlı Paralel Dağıtım Sistemleri
Proje 2: Veritabanı Yedekleme ve Felaketten Kurtarma Planı

Selim Bedirhan Öztürk
20291277

GitHub: <https://github.com/selimbedirhan/BLM4522-Ag-Tabanli-Paralel-Dagitim-Sistemleri>

ÖZET

Bu projede PostgreSQL 14 veritabanı yönetim sistemi üzerinde kapsamlı bir yedekleme ve felaketten kurtarma altyapısı tasarlanıp uygulanmıştır. Gerçek dünya senaryolarını yansıtmak amacıyla bir e-ticaret veritabanı oluşturulmuş ve 15.000'den fazla kayıt ile doldurulmuştur. Yedekleme stratejisi olarak üç katmanlı bir yapı benimsenmiştir: tam (full) yedekleme, fark (differential) yedekleme ve artık (incremental/WAL) yedekleme. Yedekleme işlemlerinin otomasyonu Bash scripting ve Cron zamanlayıcı ile sağlanmıştır. Ayrıca dört farklı felaket senaryosu simüle edilmiş, Point-in-Time Recovery (PITR) uygulanmış, yedeklerin bütünlüğü otomatik testlerle doğrulanmış ve HTML formatında raporlama sistemi kurulmuştur.

İÇİNDEKİLER

1. Giriş ve Motivasyon
2. Ortam Hazırlığı ve Kullanılan Teknolojiler
3. Veritabanı Tasarımı ve Oluşturulması
4. Örnek Veri Yükleme
5. Tam (Full) Yedekleme Stratejisi
6. Fark (Differential) Yedekleme Stratejisi
7. Artık (Incremental) Yedekleme — WAL Arşivleme
8. Zamanlanmış Otomatik Yedekleme
9. Felaketten Kurtarma Senaryoları
10. Point-in-Time Recovery (PITR)
11. Yedek Doğrulama ve Test Sistemi
12. Yedekleme Raporlama Sistemi
13. Sonuç ve Değerlendirme
14. Kaynakça

1. Giriş ve Motivasyon

Veritabanı sistemleri, modern bilgi teknolojisi altyapılarının en kritik bileşenlerinden biridir. Donanım arızaları, yazılım hataları, insan kaynaklı yanlış işlemler veya siber saldırılar gibi beklenmedik durumlar her an veri kaybına yol açabilir. İş sürekliliğinin sağlanabilmesi için verilerin düzenli olarak yedeklenmesi ve olası bir felaket durumunda hızlıca kurtarılabilmesi hayati önem taşır.

Bu projenin temel amacı, PostgreSQL veritabanı yönetim sistemi üzerinde farklı yedekleme yöntemlerini (tam, fark, artık) uygulamak, bu yedeklemeleri zamanlayıcılar ile otomatik hale getirmek ve çeşitli felaket senaryolarında veri kurtarma süreçlerini baştan sona test etmektir.

Proje kapsamında ele alınan konular şunlardır:

- Farklı formatlarda (Custom, Plain SQL, Compressed, Tar) tam yedekleme
- Şema bazlı, tablo bazlı ve tarih filtreli fark yedekleme
- WAL (Write-Ahead Logging) altyapısı ile artık yedekleme
- Cron zamanlayıcı ile günlük, haftalık ve saatlik otomatik yedekleme
- Tablo silme, veritabanı bozulma, yanlış güncelleme ve kitlesel silme senaryolarında kurtarma
- Point-in-Time Recovery (PITR) ile belirli bir zamana geri dönme
- 5 gruptan oluşan kapsamlı yedek doğrulama testleri
- Terminal ve HTML formatında otomatik raporlama

2. Ortam Hazırlığı ve Kullanılan Teknolojiler

2.1 PostgreSQL 14

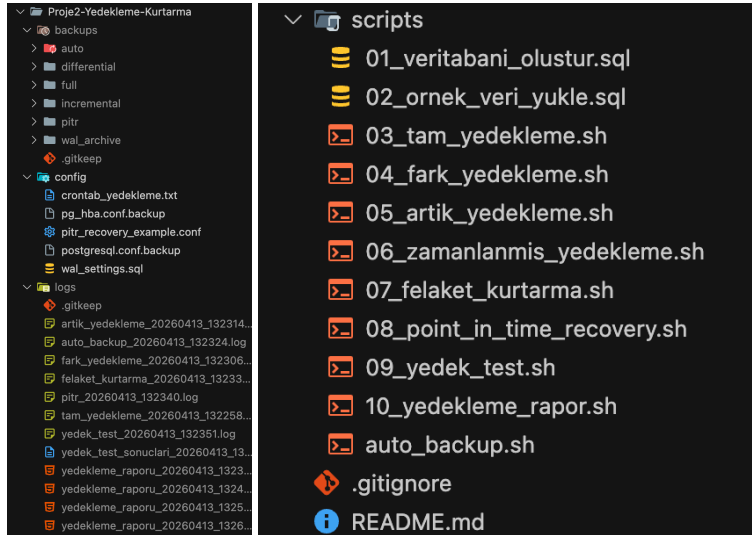
Projede veritabanı yönetim sistemi olarak PostgreSQL 14 tercih edilmiştir. PostgreSQL'in sunduğu WAL tabanlı güçlü yedekleme altyapısı, pg_dump, pg_restore ve pg_basebackup gibi zengin araç seti ve ACID uyumluluğu bu seçimin başlıca nedenleridir. İşletim sistemi olarak macOS kullanılmış, PostgreSQL Homebrew üzerinden /opt/homebrew/opt/postgresql@14/bin dizinine kurulmuştur.

2.2 Araç ve Teknoloji Tablosu

Araç / Teknoloji	Türü	Açıklama
PostgreSQL 14	VTYS	Açık kaynak ilişkisel veritabanı yönetim sistemi
pg_dump	Mantıksal Yedekleme	Veritabanını SQL komutları veya özel format olarak dışa aktarır
pg_restore	Mantıksal Yükleme Geri	pg_dump çıktısından veritabanını geri yükler
pg_basebackup	Fiziksel Yedekleme	Veri dizininin fiziksel kopyasını alır (PITR için)
WAL Arşivleme	Fiziksel	Transaction log dosyalarını sürekli arşivler
psql	SQL İstemcisi	Komut satırı üzerinden SQL sorguları çalıştırır
Bash	Scripting	Shell scripting ile otomasyon
Cron	Zamanlayıcı	Zamanlanmış görev yönetimi
gzip	Sıkıştırma	Yedek dosyalarının sıkıştırılması
bc	Hesaplama	Bash içinde matematiksel hesaplamalar

2.3 Proje Dizin Yapısı

Proje dosyaları aşağıdaki yapıda organize edilmiştir:



3. Veritabanı Tasarımı ve Oluşturulması

3.1 E-Ticaret Veritabanı Şeması

Yedekleme ve kurtarma işlemlerinin anlamlı veriler üzerinde test edilebilmesi için gerçekçi bir **e-ticaret veritabanı** tasarlanmıştır. Veritabanı `eticaret_db` adıyla oluşturulmuş ve UTF-8 karakter kodlaması kullanılmıştır.

Veritabanı oluşturma komutu:

```
10 DROP DATABASE IF EXISTS eticaret_db;
11
12 CREATE DATABASE eticaret_db
13     WITH
14     ENCODING = 'UTF8'
15     TEMPLATE = template0;
```

3.2 Tablo Yapıları

Veritabanı 8 ana tablodan oluşmaktadır. Her tablo uygun veri tipleri, NOT NULL kısıtlamaları, CHECK kontrolleri ve DEFAULT değerlerle tasarlanmıştır:

#	Tablo Adı	Açıklama	Anahtar İlişkiler
1	kategoriler	Ürün kategorileri ve alt kategoriler	Kendi kendine FK (<code>ust_kategori_id</code>)
2	tedarikciler	Tedarikçi firma bilgileri	—
3	musteriler	Müşteri iletişim ve adres bilgileri	—
4	urunler	Ürün kataloğu (fiyat, stok, durum)	FK → kategoriler, tedarikciler
5	siparisler	Sipariş kayıtları ve durumları	FK → musteriler
6	siparis_detaylari	Sipariş kalemleri ve miktarları	FK → siparisler, urunler
7	odemeler	Ödeme işlemleri ve tipleri	FK → siparisler
8	stok_hareketleri	Stok giriş/çıkış hareketleri	FK → urunler

3.3 Örnek Tablo Tanımı: siparisler

```
96 CREATE TABLE siparisler (
97     siparis_id SERIAL PRIMARY KEY,
98     musteri_id INTEGER NOT NULL REFERENCES musteriler
99     (musteri_id),
100     siparis_tarihi TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
101     teslim_tarihi TIMESTAMP,
102     durum VARCHAR(30) DEFAULT 'beklemede'
103     CHECK (durum IN ('beklemede', 'onaylandi',
104     'hazirlaniyor', 'kargoda', 'teslim_edildi',
105     'iptal')),
106     toplam_tutar DECIMAL(12, 2) DEFAULT 0,
107     kargo_ucreti DECIMAL(8, 2) DEFAULT 0,
108     indirim_tutari DECIMAL(8, 2) DEFAULT 0,
109     odeme_yontemi VARCHAR(30) CHECK (odeme_yontemi IN
110     ('kredi_karti', 'havale', 'kapida_odeme')),
111     kargo_adresi TEXT,
112     notlar TEXT,
113     olusturma_tarihi TIMESTAMP DEFAULT CURRENT_TIMESTAMP
114 );
```

3.4 İndeksler

Veritabanı performansını artırmak ve yedekleme/kurtarma sonrası bütünlük kontrollerinde kullanılmak üzere 15'ten fazla stratejik indeks tanımlanmıştır:

```
164 -- Müşteri indeksleri
165 CREATE INDEX idx_musteriler_email ON musteriler(email);
166 CREATE INDEX idx_musteriler_sehir ON musteriler(sehir);
167 CREATE INDEX idx_musteriler_kayit_tarihi ON musteriler
    (kayit_tarihi);
168
169 -- Ürün indeksleri
170 CREATE INDEX idx_urunler_kategori ON urunler(kategori_id);
171 CREATE INDEX idx_urunler_tedarikci ON urunler(tedarikci_id);
172 CREATE INDEX idx_urunler_urun_kodu ON urunler(urun_kodu);
173 CREATE INDEX idx_urunler_fiyat ON urunler(birim_fiyat);
174
175 -- Sipariş indeksleri
176 CREATE INDEX idx_siparisler_musteri ON siparisler
    (musteri_id);
177 CREATE INDEX idx_siparisler_tarih ON siparisler
    (siparis_tarihi);
178 CREATE INDEX idx_siparisler_durum ON siparisler(durum);
179
180 -- Sipariş detay indeksleri
181 CREATE INDEX idx_siparis_detaylari_siparis ON
    siparis_detaylari(siparis_id);
182 CREATE INDEX idx_siparis_detaylari_urun ON siparis_detaylari
    (urun_id);
183
184 -- Ödeme indeksleri
185 CREATE INDEX idx_odemeler_siparis ON odemeler(siparis_id);
186 CREATE INDEX idx_odemeler_tarih ON odemeler(odeme_tarihi);
187
```

-- ve diğerleri...

3.5 Görünümler (Views)

Veritabanında sık kullanılan sorguları basitleştirmek için 3 görünüm tanımlanmıştır:

- **v_siparis_ozeti**: Sipariş, müşteri ve ödeme bilgilerini bir araya getiren kapsamlı özet görünüm.
- **v_stok_durumu**: Ürün stok seviyelerini, minimum stok eşiklerini ve kritik stok uyarılarını gösteren görünüm.
- **v_gunluk_satis**: Günlük satış adetlerini ve tutarlarını raporlayan analitik görünüm.

3.6 Tetikleyiciler (Triggers) ve Fonksiyonlar

İş kurallarını veritabanı seviyesinde uygulamak için 2 tetikleyici fonksiyonu tanımlanmıştır:

- **fn_stok_guncelle()**: Sipariş detayı eklendiğinde ilgili ürünün stok miktarını otomatik olarak düşürür.
- **fn_siparis_toplam_guncelle()**: Sipariş detayı eklendiğinde siparişin toplam tutarını otomatik olarak günceller.

```

256 CREATE OR REPLACE FUNCTION fn_stok_guncelle()
257 RETURNS TRIGGER AS $$
258 BEGIN
259     IF TG_OP = 'INSERT' THEN
260         UPDATE urunler
261         SET stok_miktari = stok_miktari - NEW.miktar,
262             guncelleme_tarihi = CURRENT_TIMESTAMP
263         WHERE urun_id = NEW.urun_id;
264
265         INSERT INTO stok_hareketleri (urun_id,
266             hareket_tipi, miktar, referans_no, aciklama)
267         VALUES (NEW.urun_id, 'cikis', NEW.miktar,
268             'SIP-' || NEW.siparis_id,
269             'Sipariş ile stok düşüldü');
270     END IF;
271     RETURN NEW;
272 END;
273 $$ LANGUAGE plpgsql;
274
275 %L for Agent
276 CREATE TRIGGER trg_stok_dusur
277 AFTER INSERT ON siparis_detaylari
278 FOR EACH ROW
279 EXECUTE FUNCTION fn_stok_guncelle();

```

4. Örnek Veri Yükleme

Yedekleme ve kurtarma testlerinin gerçekçi koşullarda yapılabilmesi için veritabanı kapsamlı bir veri seti ile doldurulmuştur. Veri üretiminde PostgreSQL'in generate_series() fonksiyonu ve Türkçe isim/adres dizileri kullanılarak doğal görünümlü kayıtlar oluşturulmuştur.

Tablo	Kayıt Sayısı	Veri Üretim Yöntemi
kategoriler	20	Hiyerarşik kategoriler (üst-alt ilişkili)
tedarikciler	15	Türk tedarikçi firmaları
musteriler	500	Türkçe isim/soyisim, e-posta, şehir
urunler	200	Farklı kategorilerde ürünler
siparisler	3.000	Son 1 yıla yayılmış siparişler
siparis_detaylari	8.000	Siparişlere bağlı kalemler
odemeler	2.500	Farklı ödeme yöntemleri
stok_hareketleri	1.200+	Stok giriş/çıkış hareketleri
TOPLAM	15.000+	

Veri yükleme sonrası ANALYZE komutu çalıştırılarak PostgreSQL sorgu planlayıcısının istatistikleri güncellenmiştir.

5. Tam (Full) Yedekleme Stratejisi

Tam yedekleme, veritabanının tüm yapısını ve verilerini kapsayan en temel yedekleme türüdür. Projede dört farklı yedekleme formatı uygulanarak karşılaştırmalı analiz yapılmıştır.

5.1 Custom Format (-Fc)

```
pg_dump -U $DB_USER -d eticaret_db -Fc -Z 6 \  
--verbose --file=eticaret_full.dump
```

- **Dahili sıkıştırma** (Z 6 seviyesinde gzip) sayesinde dosya boyutu küçülür.
- **Seçici geri yükleme** desteği ile sadece belirli bir tablo veya şema restore edilebilir.
- **Paralel restore** imkânı sağlar.
- Yalnızca pg_restore ile geri yüklenebilir, doğrudan okunamaz.
- **Günlük yedeklemeler için önerilen format** olarak seçilmiştir.

5.2 Plain SQL Format (-Fp)

```
pg_dump -U $DB_USER -d eticaret_db -Fp \  
--create --clean --file=eticaret_full.sql
```

- İnsan tarafından okunabilir SQL komutları üretir.
- --create ile veritabanı oluşturma komutu dahil edilir.
- --clean ile mevcut objelerin silinmesi sağlanır.
- Farklı PostgreSQL sürümlerine taşınabilir.
- Sıkıştırma bulunmadığı için dosya boyutu büyüktür.

5.3 Sıkıştırılmış SQL Format (gzip)

```
pg_dump -U $DB_USER -d eticaret_db -Fp --create --clean \  
| gzip -9 > eticaret_full.sql.gz
```

- Plain SQL'in okunabilirliği ile gzip sıkıştırmasının alan tasarrufunu birleştirir.
- -9 parametresi ile maksimum sıkıştırma uygulanır.
- Uzun süreli arşivleme için idealdir.

5.4 Tar Format (-Ft)

```
pg_dump -U $DB_USER -d eticaret_db -Ft \
--file=eticaret_full.tar
```

- Dosya sistemi yapısında arşiv oluşturur.
- Taşınabilirlik gerektiren durumlarda kullanılır.

5.5 Format Karşılaştırma Özeti

Her yedekleme sonrası dosya boyutu, oluşturma süresi ve doğrulama durumu kaydedilmiştir. Script çalıştırıldığında terminalde renkli bir özet tablo görüntülenir:

Format	Boyut	Süre	Kullanım Alanı
Custom (.dump)	~140 KB	~80 ms	Günlük yedek, hızlı restore
Plain SQL (.sql)	~950 KB	~100 ms	Sürümler arası taşıma, denetim
Compressed (.sql.gz)	~120 KB	~120 ms	Uzun süreli arşiv
Tar (.tar)	~950 KB	~90 ms	Taşınabilirlik

6. Fark (Differential) Yedekleme Stratejisi

PostgreSQL'de SQL Server'daki gibi yerel (native) differential backup mekanizması bulunmamaktadır. Bu nedenle fark yedekleme çeşitli tekniklerle simüle edilmiştir.

6.1 Şema (DDL) Yedekleme

Sadece tablo yapılarını, indeksleri, trigger'ları ve fonksiyonları yedekler:

```
pg_dump -U $DB_USER -d eticaret_db --schema-only \
--file=sema_yedek.sql
```

6.2 Sadece Veri Yedekleme

Tablo yapıları hariç, yalnızca verileri INSERT komutları olarak dışa aktarır:

```
pg_dump -U $DB_USER -d eticaret_db --data-only \
--column-inserts --file=veri_yedek.sql
```

6.3 Tablo Bazlı Kısmi Yedekleme

Sık değişen tablolar için bireysel yedekleme yapılır. Bu yöntem tam yedeklemeye kıyasla çok daha hızlıdır:

```
pg_dump -U $DB_USER -d eticaret_db \
--table=siparisler --table=siparis_detaylari \
--file=siparisler_yedek.sql
```

6.4 Tarih Filtreli Koşullu Yedekleme

Son N günde eklenen veya değiştirilen verilerin CSV formatında dışa aktarılması:

```
140 psql -U "$DB_USER" -d "$DB_NAME" -c "\COPY (SELECT * FROM
    siparisler WHERE siparis_tarihi >= CURRENT_TIMESTAMP -
    INTERVAL '$DAYS_AGO days') TO STDOUT WITH CSV HEADER" >>
    "$BACKUP_RECENT" 2>> "$LOG_DIR/fark_yedekleme_${TIMESTAMP}.
    log"
141
```

7. Artık (Incremental) Yedekleme — WAL Arşivleme

PostgreSQL'in Write-Ahead Logging (WAL) altyapısı, gerçek anlamda artık (incremental) yedeklemeyi mümkün kılar. WAL, veritabanındaki her değişikliğin öncelikle log dosyalarına yazılması prensibiyle çalışır.

7.1 WAL Yapılandırması

WAL tabanlı sürekli yedekleme için gerekli postgresql.conf parametreleri:

```
9 wal_level = replica
10 archive_mode = on
11 archive_command = 'cp %p /path/to/wal_archive/%f'
```

Script, mevcut WAL ayarlarını kontrol eder ve gerekirse yapılandırma önerilerini ekrana yazdırır.

7.2 pg_basebackup ile Baz Yedekleme

PITR için temel teşkil eden fiziksel yedekleme pg_basebackup aracıyla alınır:

```
pg_basebackup -U $DB_USER -D /path/to/backup \
-Ft -z -P --checkpoint=fast
```

- -Ft: Tar formatında çıktı
- -z: gzip sıkıştırma
- -P: İlerleme göstergesi
- --checkpoint=fast: Hızlı checkpoint başlatma

7.3 WAL Konum Takibi

WAL yedekleme sırasında başlangıç ve bitiş LSN (Log Sequence Number) değerleri kaydedilir:

```
117 SELECT
118     pg_current_wal_lsn() as wal_konum,
119     pg_walfile_name(pg_current_wal_lsn()) as wal_dosya,
```

Bu bilgiler, bir kurtarma senaryosunda hangi WAL dosyalarının gerekli olduğunun belirlenmesinde kullanılır.

8. Zamanlanmış Otomatik Yedekleme

8.1 Otomatik Yedekleme Scripti

auto_backup.sh scripti Cron zamanlayıcı tarafından çağrılmak üzere tasarlanmıştır. Her çalıştığında aşağıdaki işlemleri gerçekleştirir:

- 1 PostgreSQL erişilebilirlik kontrolü (pg_isready)
- 2 Veritabanı varlık kontrolü (psql -c "SELECT 1")
- 3 Tarih damgalı Custom format yedekleme (pg_dump -Fc -Z 6)
- 4 Yedek doğrulama (pg_restore --list)
- 5 Eski yedeklerin temizlenmesi (7 günden eski → silme)
- 6 Eski logların temizlenmesi (30 günden eski → silme)
- 7 Durum raporlama (toplam yedek sayısı ve boyut)

8.2 Cron Zamanlayıcı Planı

Zamanlama	İşlem	Cron İfadesi
Her gün 02:00	Günlük tam yedekleme	0 2 * * *
Her Pazar 03:00	Haftalık tam yedekleme	0 3 * * 0
Hafta içi 09:00-18:00 (saatlik)	Fark yedekleme	0 9-18 * * 1-5

Cron tanımları config/crontab_yedekleme.txt dosyasına kaydedilmekte ve aşağıdaki komutla kurulmaktadır:

```
crontab config/crontab_yedekleme.txt
crontab -l # Doğrulama
```

8.3 Saklama Politikası (Retention Policy)

- **Günlük yedekler:** 7 gün saklanır, sonra find ... -mtime +7 -delete ile otomatik silinir.
- **Log dosyaları:** 30 gün sonra temizlenir.
- Temizlenen dosyalar log dosyasına kaydedilir.

9. Felaketten Kurtarma Senaryoları

Dört farklı felaket senaryosu simüle edilerek kurtarma işlemleri başarıyla gerçekleştirilmiştir.

9.1 Senaryo 1: Yanlışlıkla Tablo Silme

Problem: Bir kullanıcı yanlışlıkla DROP TABLE stok_hareketleri CASCADE komutunu çalıştırdı.

Kurtarma Süreci:

```
pg_restore -U $DB_USER -d eticaret_db \  
  --table=stok_hareketleri \  
  --single-transaction \  
  eticaret_full.dump
```

- --table parametresi ile yalnızca silinen tablo geri yüklenir.
- --single-transaction ile atomik geri yükleme sağlanır; hata durumunda tüm işlem geri alınır.
- Kurtarma sonrası kayıt sayıları karşılaştırılarak doğrulanır.

9.2 Senaryo 2: Veritabanı Tam Bozulma

Problem: Veritabanı dosyaları bozulmuş veya sistemik bir hata oluşmuştur.

Kurtarma Süreci:

```
createdb eticaret_kurtarma  
pg_restore -U $DB_USER -d eticaret_kurtarma \  
  --clean --if-exists \  
  --single-transaction \  
  eticaret_full.dump
```

Doğrulama: Orijinal ve kurtarılan veritabanının tablo tablo kayıt sayıları karşılaştırılır:

Tablo	Orijinal	Kurtarılan	Durum
musteriler	500	500	✓ EŞİT
urunler	200	200	✓ EŞİT
siparisler	3000	3000	✓ EŞİT
...			

9.3 Senaryo 3: Yanlış Veri Güncelleme

Problem: UPDATE urunler SET birim_fiyat = 0 komutu yanlışlıkla çalıştırıldı ve tüm ürün fiyatları sıfırlandı.

Kurtarma Süreci:

- 1 Yedekten geçici bir veritabanına restore yapılır.
- 2 Geçici veritabanından doğru fiyat verileri çekilir.
- 3 Ana veritabanındaki hatalı fiyatlar güncellenir.

```
238 UPDATE eticaret_db.public.urunler u
239 SET birim_fiyat = k.birim_fiyat
240 FROM eticaret_kurtarma.public.urunler k
241 WHERE u.urun_id = k.urun_id;
```

9.4 Senaryo 4: Kitleselel Veri Silme

Problem: DELETE FROM musteriler WHERE sehir = 'İstanbul' komutu çalıştırıldı. Bağımlı siparişler, sipariş detayları ve ödemeler de cascade ile silinmiştir.

Kurtarma: Tam veritabanı restore ile tüm veriler geri yüklenir. Kurtarma sonrası silinen ve geri yüklenen kayıt sayıları karşılaştırmalı olarak raporlanır.

10. Point-in-Time Recovery (PITR)

Point-in-Time Recovery, veritabanını geçmişteki belirli bir zaman noktasına geri döndürme işlemidir. Bu projede PITR süreci 6 aşamalı olarak simüle edilmiştir.

10.1 PITR Süreci

Aşama 1 — WAL Durum Kontrolü: Mevcut wal_level ve archive_mode ayarları kontrol edilir.

Aşama 2 — Zaman Noktası Kaydı: SELECT CURRENT_TIMESTAMP ile hedef geri dönüş zamanı kaydedilir. pg_create_restore_point('pitr_test_TIMESTAMP') ile restore point oluşturulur ve bu noktanın yedeği alınır.

Aşama 3 — Felaket Simülasyonu: Üç farklı zararlı işlem uygulanır:

- 50 müşteri ve bağımlı kayıtları silinir
- Tüm ürün fiyatları %50 artırılır
- 200 adet sahte sipariş eklenir

Aşama 4 — PITR Kurtarma: Baz yedekten test veritabanına restore yapılır:

```
pg_restore -U $DB_USER -d eticaret_pitr_test \
--clean --if-exists \
--single-transaction \
eticaret_pitr_base.dump
```

Aşama 5 — Doğrulama: Kurtarılan ve bozulmuş veritabanı tablo tablo karşılaştırılır. Ortalama ürün fiyatları kontrol edilir.

Aşama 6 — Ana Veritabanını Kurtarma: Kurtarılan veritabanından alınan yedek ile ana veritabanı felaket öncesi durumuna geri getirilir.

10.2 PITR Yapılandırma Parametreleri

PostgreSQL 12+ sürümlerinde postgresql.conf'a eklenmesi gereken PITR yapılandırma parametreleri:

```
recovery_target_time = '2026-04-13 12:00:00'  
recovery_target_action = 'promote'  
recovery_target_inclusive = true  
restore_command = 'cp /path/to/wal_archive/%f %p'  
recovery_target_timeline = 'latest'
```

11. Yedek Doğrulama ve Test Sistemi

Yedeklerin güvenilirliğini kanıtlamak için 5 test grubundan oluşan kapsamlı bir doğrulama sistemi uygulanmıştır.

11.1 Test Grubu 1: Yedek Oluşturma Testleri

- Custom format (.dump) yedeği başarıyla oluşturulabilmeli
- Plain SQL (.sql) yedeği başarıyla oluşturulabilmeli
- Her iki dosyanın boyutu sıfırdan büyük olmalı

11.2 Test Grubu 2: Bütünlük Kontrolleri

- pg_restore --list ile yedek içerik listesi alınabilmeli
- Dosya boyutu kontrolü (sıfır olmadığından emin olma)
- SQL dosyasında CREATE TABLE komutlarının varlığı doğrulanmalı

11.3 Test Grubu 3: Geri Yükleme Testleri

- Test veritabanına (eticaret_test_restore) restore yapılabilmesi
- Restore edilen veritabanının kayıt sayıları orijinal ile eşleşmeli

11.4 Test Grubu 4: Veri Bütünlüğü Testleri

- **Foreign Key ilişkileri:** Yetim kayıt olmamalı (orphan records)
- **İndeksler:** Tüm tanımlı indeksler mevcut olmalı
- **View'lar:** Tüm görünümeler çalışır durumda olmalı
- **Trigger ve fonksiyonlar:** Tüm fonksiyonlar sağlam olmalı
- **Veri doğruluğu:** Toplam tutar ve ortalama fiyat kontrolleri

11.5 Test Grubu 5: Performans Testleri

- Yedekleme süresi ölçümü
- Veritabanı boyutu ve yedek boyutu oranı (sıkıştırma verimliliği)

11.6 Test Sonuç Formatı

Her test GEÇTİ veya KALDI olarak raporlanır ve detaylar log dosyasına yazılır:

Test 1.1: Custom format yedek oluşturma GEÇTİ

Test 1.2: Plain SQL yedek oluşturma GEÇTİ

Test 2.1: pg_restore --list kontrolü GEÇTİ

Test 2.2: Dosya boyutu kontrolü GEÇTİ

...

Toplam: 15/15 test başarılı

12. Yedekleme Raporlama Sistemi

12.1 Terminal Raporu

10_yedekleme_rapor.sh scripti çalıştırıldığında 5 bölümden oluşan bir terminal raporu sunar:

- 1 **Veritabanı Genel Bilgileri:** Tüm veritabanlarının ve tabloların boyutları
- 2 **Yedek Dosyaları Listesi:** Full, differential ve otomatik yedek dosyaları (isim, boyut, tarih)
- 3 **Yedekleme İstatistikleri:** Toplam yedek sayısı ve boyut
- 4 **PostgreSQL Durum Bilgileri:** WAL Level, Archive Mode, Max Connections, Shared Buffers
- 5 **HTML Rapor Oluşturma:** Dark theme tasarımlı HTML rapor

12.2 HTML Rapor

Modern ve görsel açıdan zengin bir HTML rapor otomatik olarak oluşturulur. Rapor aşağıdaki bileşenleri içerir:

- **Metrik Kartları:** Veritabanı boyutu, toplam yedek sayısı, yedek boyutu
- **Tablo Detayları:** Her tablonun kayıt sayısı ve boyutu
- **Yedek Dosyaları Tablosu:** Tüm yedeklerin listesi, boyut ve durumu
- **PostgreSQL Yapılandırma:** WAL Level, Archive Mode gibi kritik parametreler
- Dark theme ile modern tasarım

HTML rapor logs/ dizinine kaydedilir ve open komutu ile tarayıcıda görüntülenebilir.

13. Sonuç ve Değerlendirme

13.1 Elde Edilen Kazanımlar

Bu proje kapsamında aşağıdaki kazanımlar sağlanmıştır:

- PostgreSQL'in yedekleme araçlarının (pg_dump, pg_restore, pg_basebackup) farklı senaryolarda etkin kullanımı
- Dört farklı yedekleme formatının (Custom, Plain SQL, Compressed, Tar) avantaj ve dezavantajlarının karşılaştırmalı analizi
- WAL tabanlı sürekli yedekleme altyapısının kavranması ve uygulanması
- Cron zamanlayıcı ile otomatik yedekleme politikalarının oluşturulması
- Dört gerçekçi felaket senaryosunda başarılı veri kurtarma
- PITR ile belirli bir zaman noktasına geri dönme yetkinliği
- Shell scripting ile veritabanı otomasyon becerileri

13.2 En İyi Uygulamalar (Best Practices)

- 1 **3-2-1 Kuralı:** En az 3 yedek kopya, 2 farklı ortamda, 1 tanesi offsite
- 2 **Düzenli Test:** Yedeklerin düzenli olarak restore testi yapılmalı
- 3 **Otomasyon:** Manuel yedekleme yerine zamanlanmış otomatik yedekleme kullanılmalı
- 4 **İzleme:** Yedekleme işlemlerinin başarı durumu sürekli izlenmeli
- 5 **Saklama Politikası:** Eski yedeklerin otomatik temizlenmesi planlanmalı
- 6 **Dökümantasyon:** Kurtarma adımları detaylı şekilde belgelenmeli

13.3 RPO ve RTO Analizi

- **RPO (Recovery Point Objective):** WAL arşivleme aktifken neredeyse sıfır veri kaybı sağlanabilir. Günlük tam yedekleme ile maksimum 24 saatlik veri kaybı riski söz konusu olur.
- **RTO (Recovery Time Objective):** Yedek boyutuna bağlı olarak dakikalar ile saatler arasında değişir. Custom format ile restore, Plain SQL'e kıyasla önemli ölçüde daha hızlıdır.

14. Kaynakça

- 1 PostgreSQL Documentation — Backup and Restore
<https://www.postgresql.org/docs/14/backup.html>
- 2 PostgreSQL Documentation — Continuous Archiving and Point-in-Time Recovery
<https://www.postgresql.org/docs/14/continuous-archiving.html>
- 3 PostgreSQL Documentation — pg_dump <https://www.postgresql.org/docs/14/app-pgdump.html>
- 4 PostgreSQL Documentation — pg_restore <https://www.postgresql.org/docs/14/app-pgrestore.html>
- 5 PostgreSQL Documentation — pg_basebackup <https://www.postgresql.org/docs/14/app-pgbasebackup.html>
- 6 PostgreSQL Documentation — Write-Ahead Logging (WAL)
<https://www.postgresql.org/docs/14/wal.html>
- 7 PostgreSQL Documentation — Recovery Configuration
<https://www.postgresql.org/docs/14/recovery-config.html>
- 8 Linux man page — crontab(5) <https://man7.org/linux/man-pages/man5/crontab.5.html>